One-Dimensional Array in Java

A **one-dimensional array** in Java is a collection of variables of the same type stored in a contiguous memory location. It is essentially a list of elements, and each element is accessed using an index. The index starts from `0` and goes up to `n-1` where `n` is the number of elements in the array.

Declaration and Initialization

In Java, you can declare and initialize a one-dimensional array in the following ways:

1. Declaration without Initialization

```
// Declare an array of integers
int[] arr;
```

2. Declaration with Initialization

```
// Declare and initialize an array of integers
int[] arr = new int[5];  // Array of 5 integers, initialized to default values (0)
```

3. Declaration with Initialization of Elements

```
// Declare and initialize an array with specific values
int[] arr = {1, 2, 3, 4, 5};  // Array of 5 integers with values
```

Accessing Array Elements

The individual elements of an array can be accessed using the index. Array indices in Java start from `0`.

Example of Accessing Array Elements:

```
public class OneDimensionalArray {
    public static void main(String[] args) {
        // Declare and initialize an array with values
        int[] arr = {10, 20, 30, 40, 50};

        // Accessing elements using index
        System.out.println("First element: " + arr[0]);  // Output: 10
        System.out.println("Second element: " + arr[1]); // Output: 20
```

```
        System.out.println("Third element: " + arr[2]);  // Output: 30
    }
}
```

- **Output:**

```
First element: 10
Second element: 20
Third element: 30
```

Array Length

To get the length of an array (i.e., the number of elements it can hold), you can use the `length` property of the array.

Example:

```
public class ArrayLength {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};  // Array of 5 elements
        System.out.println("Array length: " + arr.length);  // Output: 5
    }
}
```

- **Output:**

```
Array length: 5
```

Looping Through a One-Dimensional Array

To iterate through the elements of a one-dimensional array, you can use a `for` loop or an enhanced `for` loop (for-each loop).

Using a Regular `for` Loop:

```
public class ArrayLoop {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};

        // Using a regular for loop to iterate through the array
        for (int i = 0; i < arr.length; i++) {
            System.out.println("Element at index " + i + ": " + arr[i]);
        }
```

```
    }
}
```

- **Output:**

```
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50
```

Using an Enhanced `for` Loop:

```
public class EnhancedForLoop {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};

        // Using an enhanced for loop (for-each) to iterate through the
array
        for (int element : arr) {
            System.out.println(element);
        }
    }
}
```

- **Output:**

```
10
20
30
40
50
```

Example: Complete Program using a One-Dimensional Array

```
public class OneDimensionalArrayExample {
    public static void main(String[] args) {
        // Declare and initialize the array
        int[] arr = {5, 10, 15, 20, 25};

        // Printing array elements using a regular for loop
```

```
        System.out.println("Using regular for loop:");
        for (int i = 0; i < arr.length; i++) {
            System.out.println("Element at index " + i + ": " + arr[i]);
        }

        // Printing array elements using an enhanced for loop
        System.out.println("\nUsing enhanced for loop:");
        for (int element : arr) {
            System.out.println(element);
        }

        // Accessing specific elements using index
        System.out.println("\nAccessing specific elements:");
        System.out.println("Element at index 2: " + arr[2]);
        System.out.println("Element at index 4: " + arr[4]);

        // Array length
        System.out.println("\nArray length: " + arr.length);
    }
}
```

- **Output:**

```
Using regular for loop:
Element at index 0: 5
Element at index 1: 10
Element at index 2: 15
Element at index 3: 20
Element at index 4: 25

Using enhanced for loop:
5
10
15
20
25

Accessing specific elements:
Element at index 2: 15
Element at index 4: 25

Array length: 5
```